
Day 7 – Enums in Swift

What I Learned

- Learned how to create Enums using `enum`
- Understood that Enums represent a fixed set of possible values
- Used Enums with `if` conditions
- Used `switch` statements with Enums
- Learned about Raw Values in Enums
- Added methods inside Enums
- Understood the difference between Structs and Enums
- Built real-world examples like Traffic Lights, Login States, Themes, and Game States

Basic Enum Example

```
enum TrafficLight {
    case green
    case yellow
    case red
}

var signal = TrafficLight.red

print(signal)
```

Why Use Enums?

Enums make code safer by allowing only predefined values.

Without Enum

```
var status = "reddddd"
```

With Enum

```
signal = .red
```

Enum with If Condition

```
enum Subscription {
    case free
    case premium
}

let user = Subscription.premium
```

```
if user == .premium {
    print("Welcome Premium User")
}
```

◆ Switch with Enum

```
enum Direction {
    case north
    case south
    case east
    case west
}

let playerMove = Direction.north

switch playerMove {

case .north:
    print("Move UP")

case .south:
    print("Move DOWN")

case .east:
    print("Move LEFT")

case .west:
    print("Move RIGHT")
}
```

◆ Enum with Raw Values

```
enum WeekDay: String {
    case monday = "MON"
    case tuesday = "TUE"
    case wednesday = "WED"
}
```

Usage:

```
print(WeekDay.monday.rawValue)
```

◆ Enum Methods

Enums can also contain functions.

```
enum GameState {

    case start
    case pause
    case gameOver

    func message() {
```

```
        switch self {  
            case .start:  
                print("Game Started")  
  
            case .pause:  
                print("Game Paused")  
  
            case .gameOver:  
                print("Game Over")  
        }  
    }  
}
```

◆ Important Concept – `self`

Inside an Enum:

`self`

means:

Current enum value

◆ Enum vs Struct

Enum	Struct
Fixed possible values	Custom object
Best for states	Best for models
Uses <code>case</code>	Uses properties
Limited choices	Stores data

◆ Real App Examples

```
enum Theme {  
    case light  
    case dark  
}  
  
enum VideoQuality {  
    case low  
    case medium  
    case high  
}
```

◆ Practice Tasks

```
enum Planet {  
    case earth  
    case mars  
}
```

```
        case jupiter
    }
    enum GameLevel {
        case easy
        case medium
        case hard
    }
```

Important Interview Questions & Answers

1 What is an Enum in Swift?

Enum is a custom type that defines a group of related fixed values.

2 Why do we use Enums?

Enums improve type safety and prevent invalid values.

3 What is the difference between Enum and Struct?

Enums represent fixed states or choices.
Structs store related data and properties.

4 Can Enums have methods?

Yes. Enums can contain functions and computed properties.

5 What are Raw Values in Enums?

Raw values are predefined constant values associated with Enum cases.

Example:

```
enum WeekDay: String {
    case monday = "MON"
}
```

Why are Enums important in iOS development?

Enums are heavily used in:

- App states
 - Navigation
 - Network responses
 - Theme modes
 - UI states
 - API handling
-

Quick Revision

- Enum = Fixed possible values
 - `case` defines options
 - Enums improve safety
 - `switch` works great with Enums
 - Enums can have methods
 - Enums can have raw values
 - `self` means current Enum value
-

Day 7 Summary

Today I learned:

- Enum creation
 - Enum safety
 - Switch with Enums
 - Raw values
 - Enum methods
 - Real-world Enum usage
 - Enum vs Struct
-

Additional Interview Questions – Enums in Swift

What is the advantage of using Enums over Strings?

Enums provide:

- Type safety
- Better readability
- Prevent invalid values
- Cleaner switch handling

✗ String Example

```
var status = "reddddd"
```

✓ Enum Example

```
enum TrafficLight {  
    case red  
    case yellow  
    case green  
}
```

8 Why are Enums considered type-safe?

Because Enums only allow predefined cases.

Example:

```
signal = .red
```

Invalid values are not allowed.

9 What is `switch` used for with Enums?

`switch` is used to handle all possible Enum cases cleanly.

Example:

```
switch signal {  
  
case .red:  
    print("Stop")  
  
case .yellow:  
    print("Wait")  
  
case .green:  
    print("Go")  
}
```

10 Can Enums have functions in Swift?

Yes. Enums can contain methods and computed properties.

Example:

```
enum GameState {  
  
    case start  
    case pause  
  
    func message() {  
        print("Game Running")  
    }  
}
```

1 1 What are Raw Values in Enums?

Raw values are predefined constant values attached to Enum cases.

Example:

```
enum WeekDay: String {  
  
    case monday = "MON"  
    case tuesday = "TUE"  
}
```

Usage:

```
print(WeekDay.monday.rawValue)
```

1 2 Can Enums store associated values?

Yes. Enums can store additional data using associated values.

Example:

```
enum LoginResult {  
  
    case success(String)  
    case failure(String)  
}
```

Usage:

```
let result = LoginResult.success("Welcome Adarsh")
```

1 3 What is the difference between Raw Values and Associated Values?

Raw Values	Associated Values
Fixed predefined value	Dynamic value
Same for every case	Different for each instance
Stored automatically	Passed during runtime

1 4 What does `self` mean inside an Enum?

`self` refers to the current Enum case.

Example:

```
switch self {
case .start:
    print("Game Started")
}
```

1 5 Where are Enums used in real iOS apps?

Enums are commonly used in:

- Theme modes
 - Navigation states
 - API response states
 - Loading states
 - Login states
 - Network handling
 - User permissions
-

★ Pro Interview Tip

Most interviewers expect:

- Enum basics
- Switch with Enum
- Raw values
- Associated values
- Enum vs Struct vs Class

These are very commonly asked in iOS interviews.

